SC2002
Turn-based Combat Group Assignment

Tutorial Group: FCSB
Group Number: Group 6
Group Members:

1. THIAN ZHI XIN JABEZ (U2521525K)
2. TIMOTHY LOKE JUN KIT (U2522376L)
3. ZHANG HAOYUAN (U2521172F)
4. ZHANG YUESHEN (U2520977L)
5. TAN HONG WEI

Content Page

UML Class Diagram **2**
UML Sequence Diagram **5**
Design considerations and OO concepts in our design **9**
Applying SOLID Principles 9
Applying Core Object-Oriented Pillars 11
Applying Boundary Control Entity 12
Our Reflection **13**
Difficulties faced: 13
What we have learnt: 13
Future Improvements: 14
Github Link **15**
Game Test Cases **16**
1. Damage Comparison: Normal vs. Defend 16
2. Shield Bash Stun 16
3. Potion Healing 16
4. Smoke Bomb Damage Negation 17
5. Wizard Arcane Blast – Attack Increase on Kill 18
6. Skill Cooldown 18
7. Backup Wave Spawn 19
Declaration Forms **20**

UML Class Diagram

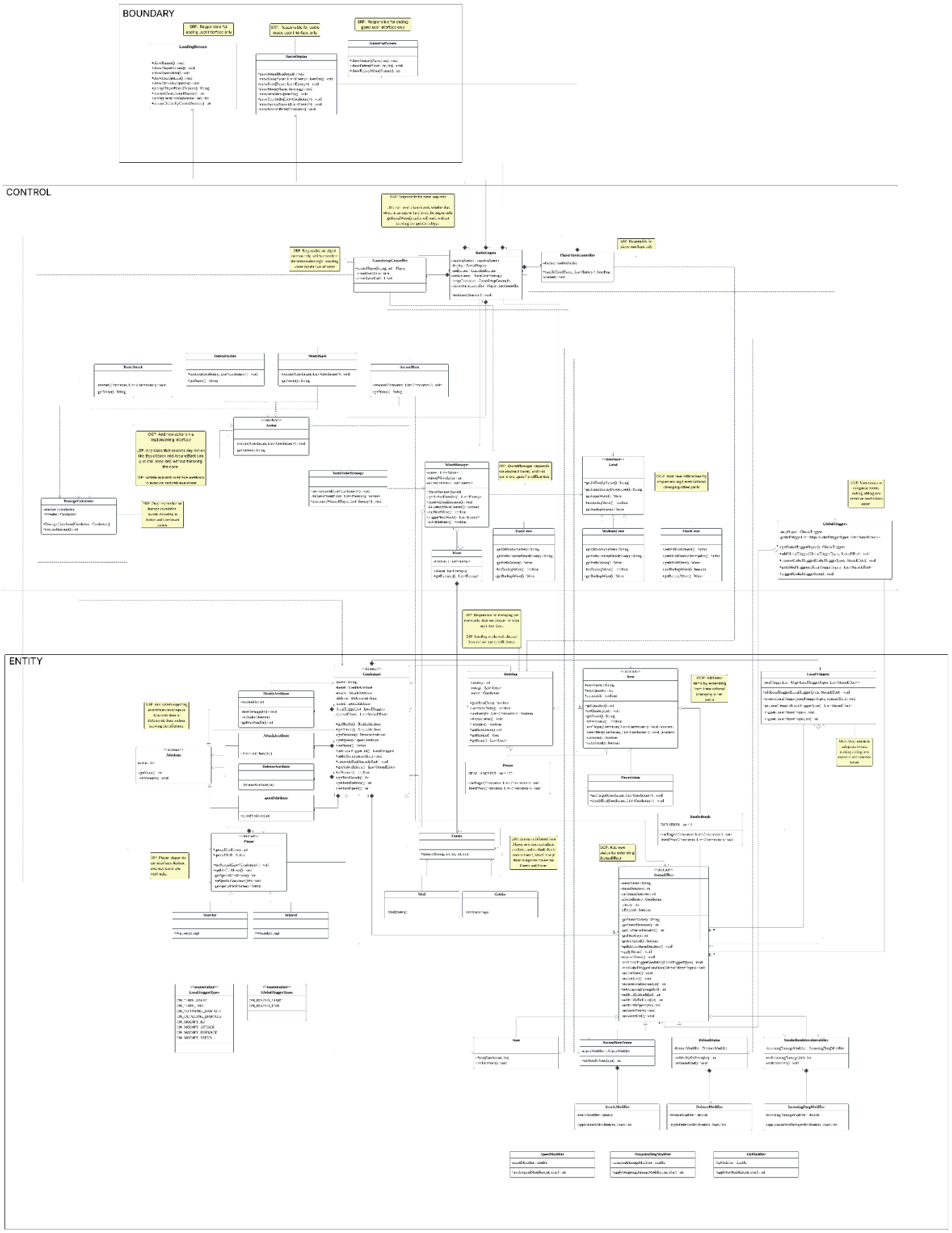

Figure 1.1 (UML Diagram)

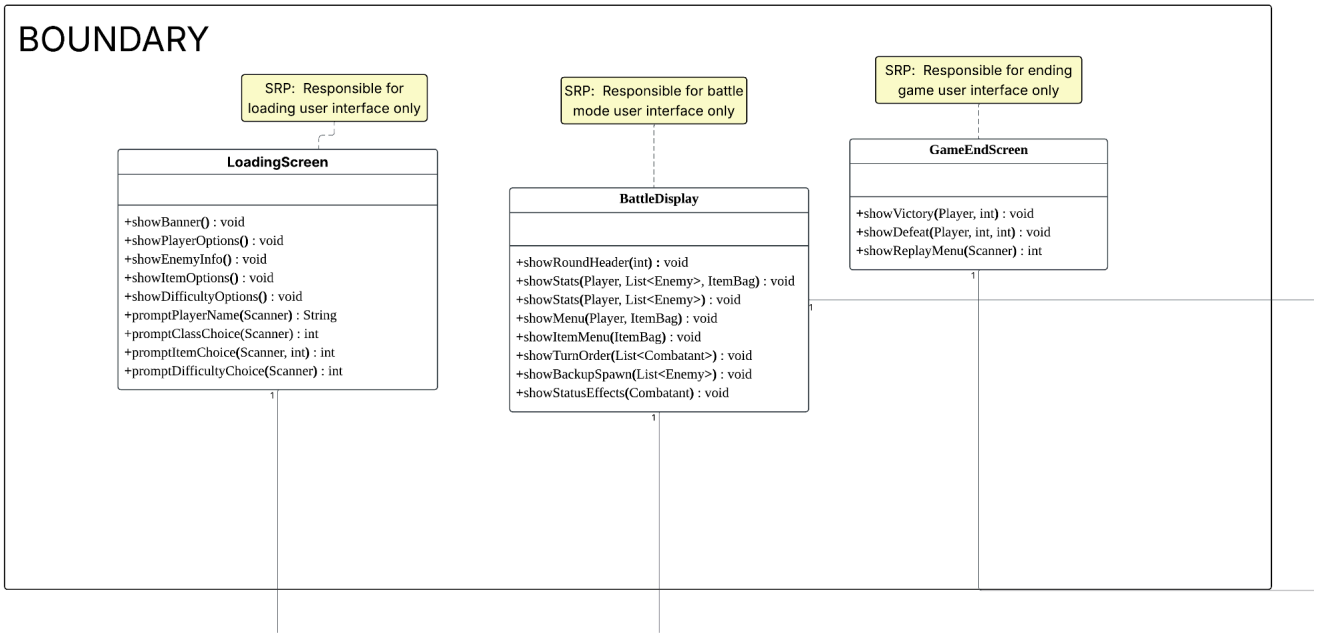

Figure 1.2 (Boundary Close-up)

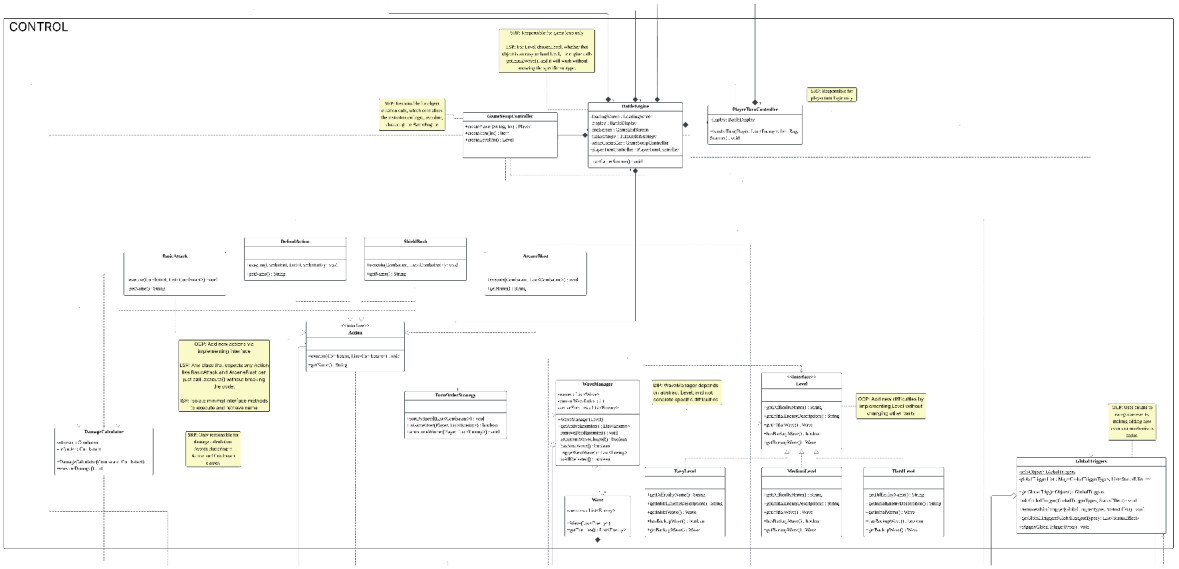

Figure 1.3 (Control Close-up)

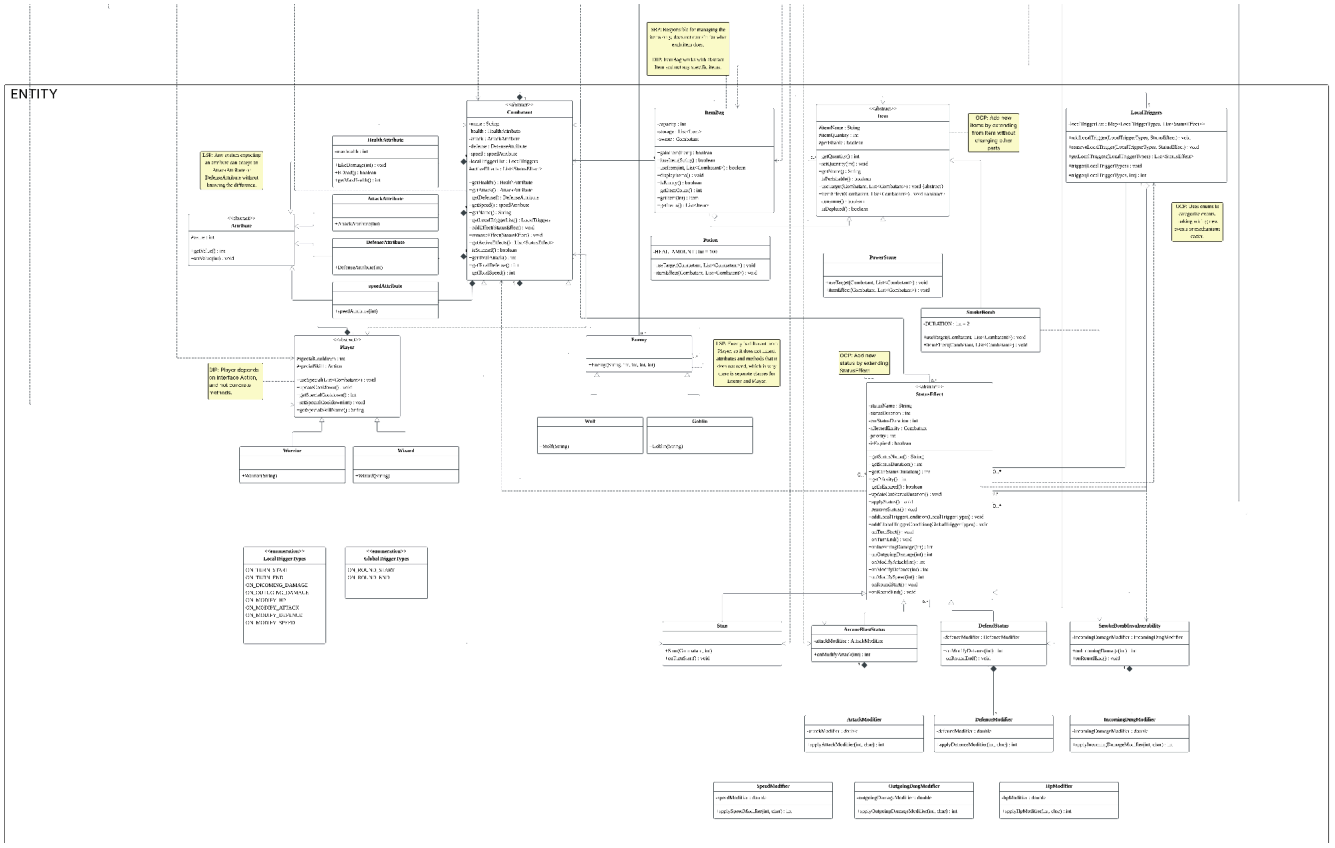

Figure 1.4 (Entity Close-up)

UML Sequence Diagram

Sequence diagram (Enemy Basic Attack)

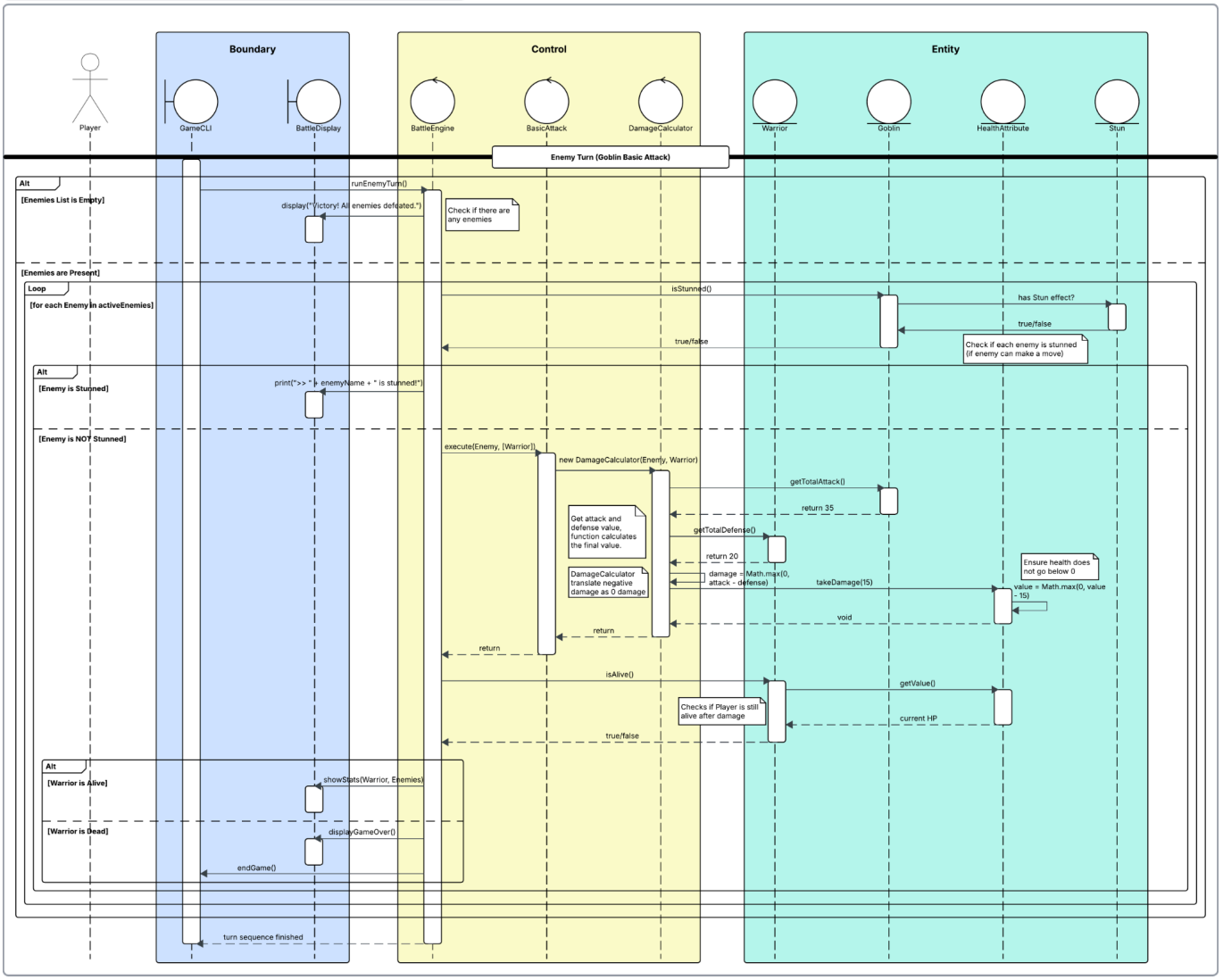

Figure 2 (Sequence Diagram on enemy attack)

Figure 2 is the sequence diagram for the process when it is the enemy's turn. This sequence diagram encompasses the following:

1. When it is the enemy's turn, it checks if there are any enemies left in the list. If there is no enemy, it displays victory; otherwise, it continues
2. Enemies are checked for stun effects. If stun is true, the enemy's turn is skipped; otherwise, it continues
3. The enemy does a basic attack on the player; the logic for damage calculation is handled by the damage calculator.
4. When it is the end of the enemy's turn, it checks if the player is still alive. If the player has 0 hp, it ends the game; otherwise, it continues.

Sequence diagram (Player Defense)

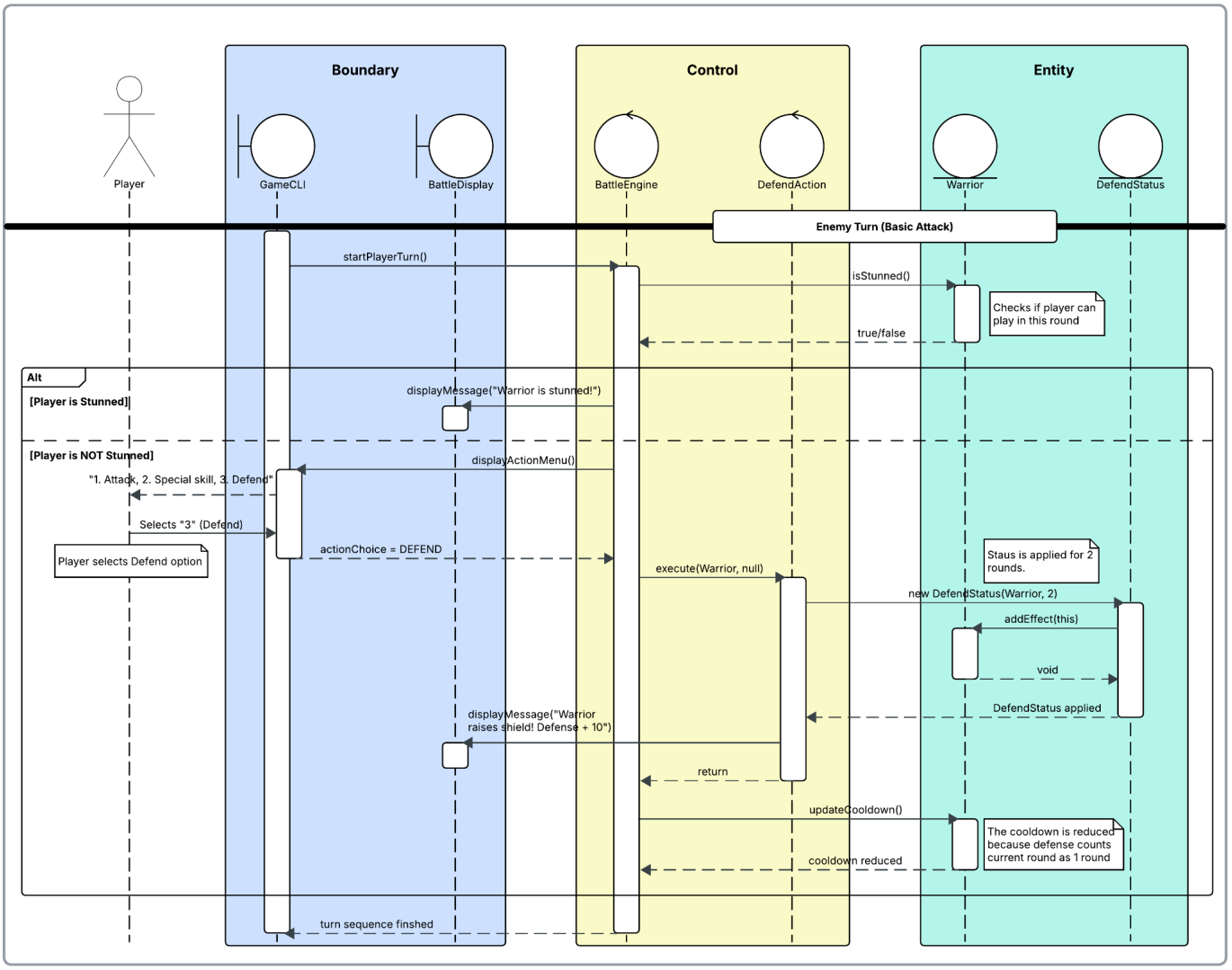

Figure 3 (Sequence Diagram on the player selecting defend)

Figure 3 is the sequence diagram for the process when it is the player's turn. This sequence diagram encompasses the following:

1. When it is the player's turn, it checks if the player is stunned. If the player is stunned, it displays a message and skips the turn; Otherwise, it continues.
2. The player can select an action, and the player selects defend

Sequence diagram (Player use Potion)

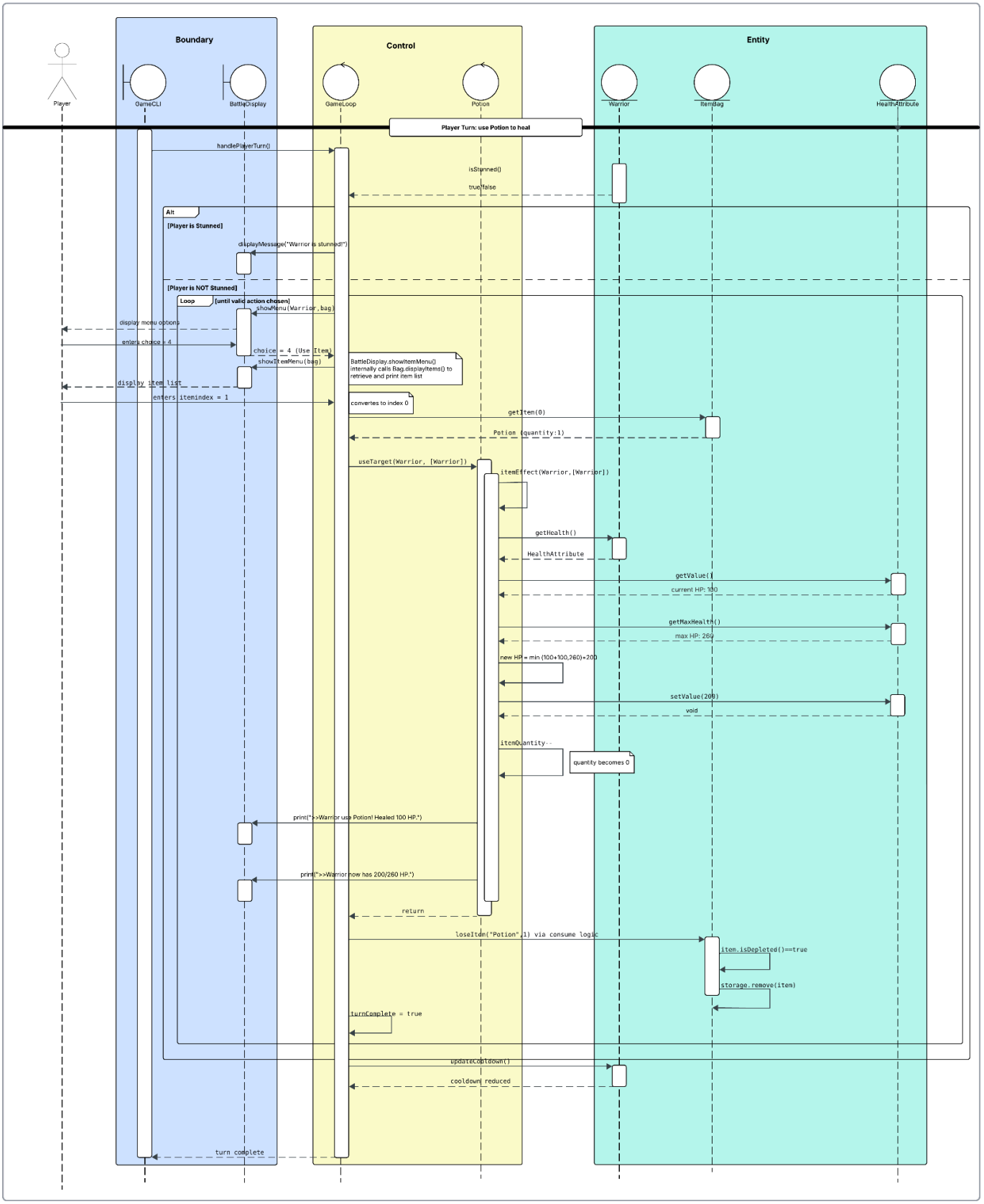

Figure 4 (Sequence Diagram on the player selecting potion to heal)

Figure 4 is the sequence diagram for the process when the player uses a Potion. This sequence diagram encompasses the following:

1. When it is the player's turn, it checks if the player is stunned. If the player is stunned, it displays a message and skips the turn; Otherwise, it continues.
2. The player selects the "Use Item" action. The system retrieves the player's bag and displays the list of available items.
3. The player chooses a Potion. The Potion's effect (heal 100 HP) is applied to the player, and the item quantity is decreased.
4. After healing, the battle display is updated to show the player's new health. The turn ends normally.

Design considerations and OO concepts in our design

Applying SOLID Principles

Single Responsibility Principle (SRP)

- ShieldBash.java deals damage and stuns. It does not handle the user interface or health points directly.
- We have multiple attribute classes that handle only one stat. An example HealthAttribute manages damage/death logic, while we have a base attribute.
- BattleEngine shows SRP as it is the high-level game loop and flow. It does not handle any damage or calculation; it only runs the system.
- DamageCalculator only handles the calculation of attack, defense and triggers. This is to avoid cluttering in the Combatant or Action classes.
- WaveManager is created to manage only the cycle of enemy groups. This allows us to change the enemy spawn by only editing this file.
- GameSetupController centralises the instantiation logic, avoiding cluttering the BattleEngine.
- ItemBag is only responsible for managing the items. It does not contain any logic for what each item does. Dedicating this class to only be an action to be used in the bag.
- We created a UI folder with different displays so that when we need to change how player stats are formatted, we can access BattleDisplay and not need to go into the GameCLI file.

Open/Closed Principle (OCP)

- In the actions file, we do not need to change the Action.java or Combatant class. We can simply create a new file. For example, we want fly as an action, we can add in Fly.java that implements Action.
- Attribute class is the base class which is “closed” for modification but “open” for extensibility. We can add in for example, speedAttribute and DefenseAttribute without editing the original Attribute code.
- We have a Level interface in the difficulty package, having to add in another difficulty such as “super easy mode” will just require to create a new java file that is implementing level. Nothing have to be changed such as the BattleEngine.java or WaveManager.java.
- Item is an abstract class which fulfills OCP as we can add in other items such as Revive by creating new classes that extends from Item. We are not required to change ItemBag or PlayerTurnController.
- The trigger system uses Enums(LocalTriggerTypes, GlobalTriggerTypes) to categorise events. We can add new game mechanics by adding a new Enum constant and a hook in the DamageCalculator. We do not need to rewrite the core such as the Combatant or Action classes.

Liskov Substitution Principle

- The actions package has any class that expects an Action can take a DefendAction or an ArcaneBlast and call .execute() without breaking the code.
- All the stats inherit from Attribute, any system expecting an attribute can accept an AttackAttribute or DefenseAttribute without knowing the difference.
- In BattleEngine, we use Level chosenLevel. Whether that object is an easy or hard level, the engine calls getInitialWave(), and it will work without knowing the specific subtype.

- The effects package contains StatusEffect class, which is abstract. Stun, DefendStatus and ArcaneBlastStatus all inherit from the status Effect. The is so that the Combatant class only maintains a List<StatusEffect>. It does not need to know if an effect is a “Stun” or something else. It just iterates through the list and calls the trigger methods. This is possible because all subclasses follow the base class, allowing the program’s behaviour to remain consistent.
- We have a hierarchy of Combatant → Player → Warrior which shows LSP. The BattleEngine treats Warrior and Wizard as Player and calls player.useSpecial() without needing to know if it is a ShieldBash or ArcaneBlast. This is because Warrior and Wizard do not change the meaning of the methods in Player.

Interface Segregation Principle (ISP)

- In the Action interface, there are only execute() and getName(). This ensures that even a non-damaging action like DefendAction isn’t forced to implement unnecessary “damage” methods.

Dependency Inversion Principle (DIP)

- BattleEngine depends on the level interface, but not specifically on the EasyLevel or HardLevel classes
- DamageCalculator does not depend on specific statuses like DefendStatus. Instead, both depend on the LocalTriggerTypes abstraction and the StatusEffect base class. This decouples the calculation from the status effects. We can add new types of statuses without changing the code in DamageCalculator.
- The player depends on the Action interface for its specialSkill. We pass Action into the super constructor which creates a class that uses DefendAction as a special skill without writing any new logic.
- ItemBag stores a List<Item>, which is an abstraction. The bag does not depend on concrete classes like Potion or SmokeBomb but instead on the abstract Item Class

Applying Core Object-Oriented Pillars

Encapsulation

- The Attribute System: We created Attribute hierarchy (HealthAttribute, AttackAttribute) instead of using primitive integers for stats (e.g. int hp).
- Data Hiding: We mark fields like value in the Attribute class or specialCooldown in the Player class as protected or private.
- Reason: This prevents any illegal status. An example, the HealthAttribute contains the logic to ensure HP never drops below zero or exceeds maxHealth. By encapsulating this logic, the rest of the program does not need to check if it is breaking the rules of the health aspect.

Abstraction

- Interfaces: The Action and Level interfaces are just abstractions.
- Simplified interfaces: The BattleEngine interacts with the level interface. It then calls getInitialWave(), but does not know if the wave contains easy or hard level (3 goblins, etc).
- Reason: This reduces the cognitive load as when we write the logic for BattleEngine, we do not need to see the complexity of the calculation inside ArcaneBlast but instead know that every Action has an execute() method.

Inheritance

- Combatant Hierarchy: Combatant(abstract) → Player and Enemy(abstract) → Warrior, Wizard and Goblin.
- All combatants need a LocalTriggers list and basic attributes. By placing all of these into the combatant as the base class, it prevents me from needing to write the same code multiple times.
- This ensures consistency and if we decide that all characters in the game should have another stat, we only add it to the Combatant class once and now every sub-class will inherit it automatically.

Polymorphism

- Method overriding: Both Warrior and Wizard are Player objects, but they override how their special skill is initiated
- In the TurnOrderStrategy, we have a List<Combatant>. When we call c.getSpeed().getValue(), Java automatically determines at runtime whether c is a Wolf or a Warrior.
- The PlayerTurnController holds a reference to an Action. Hence, it does not know if action is a ShieldBash or a BasicAttack; it just triggers execute().
- This provides Extensibility. We can add in new classes as long as it extends Player. The BattleEngine and BattleDisplay will treat it perfectly without needing to change the code.

Composition over inheritance

- Implementation:
    - A Combatant has a LocalTriggers list
    - A Player has an Action (Special Skill)
    - A Wave has a List<Enemy>
- The use of composition over inheritance makes the code more flexible. This allows us to give a warrior a second special skill we we wanted to by giving it List<Action>, instead of creating a complex inheritance tree.

Applying Boundary Control Entity

Boundary

- Implementation:
  - The ui package: BattleDisplay, LoadingScreen and GameEndScreen
  - Scanner: user input
- How it works: BattleDisplay contains all the System.outprintf logic. It takes data from the Entities and formats it into the “Party Status” table
- Why we use it: The Combatant class should never have a System.out.println inside it. If we want to move this game to a mobile app or web browser, we would need to replace the boundary classes while the rest of the engine remains exactly the same.

Control

- Implementation:
  - The core package: BattleEngine, PlayerTurnController, and TurnOrderStrategy
  - The core.DamageCalculator: A specialised controller for the calculation of an interaction
- How it works:
  - The boundary (LoadingScreen) asks the user for a name
  - The Controller (BattleEngine) takes the name and tells the Entity factory (GameSetupController) to create a new Player object
  - During the combat, PlayerTurnController takes a choice from the Boundary and tells the entities to execute an action
- Why we use it:
  - Controllers prevent having “God Classes”. Without them, the Player class would have to handle its own input and its own display, making it massive and difficult to debug. Controllers help keep the logic for “How the game runs” and separate from “What the game is”.

Entity

- Implementation:
  - Entities package (Combatant, Player, Warrior and Enemy)
  - Attributes package (HealthAttribute, AttackAttribute, etc)
- How it works:
  - These classes represent the state of the game. An example, a Warrior knows its HP and attack power.
  - The logic within these classes is strictly about the entity itself only. An example, HealthAttribute.takeDamage(), ensures HP doesn’t go below zero.
- Why we use it:
  - Isolating data into entities, we can ensure that if the rules of the Warrior were to change, we only need to change the Warrior class, and it would not affect the UI or turn logic.

Our Reflection

Difficulties faced:

There were difficulties in trying to integrate all of our codes, as we realised too late that everyone had different ideas in terms of coding the parts they were in charge of. Coding our allocated parts, it was difficult to visualise the end and also how the code would run. Thus, there were a lot of logical errors since our portions often relied on others' parts of the code, which we were unclear about. It led to some differences in logic, and we had to spend time editing and piecing everything together. So we sat down and took time to slowly untangle everything and merge it cohesively.

There were difficulties in trying to implement the status effects, as it was difficult trying to find an efficient design to update the duration and remove them since each of them behaves differently. To overcome this, we researched more Java implementations to learn more about what we could try to implement. Eventually, we learnt of enums and maps, which made trying to implement status effects easier, more structured and simpler.

We faced difficulty in trying to come up with a SOLID design. While we were touching up on our diagrams, we realised we had some improvements to make. Going back to edit the code proved to be a challenge as some of the code was interwoven with one another, making it quite problematic to change some parts of the code without the need to change another. We revised the code and double-checked it to see where we could apply more SOLID designs, cross-referencing with lecture notes and researched some common pitfalls to avoid them.

There were some difficulties in using packages to make the classes easier to see, instead of being everywhere. We were quite unfamiliar with Java at the time, and we conquered it through research and trial by fire, learning how to properly organise our code effectively.

What we have learnt:

We learned about many Java implementations that make coding easier, such as maps and enums, from trying to figure out how to implement certain aspects of the game.
We have also learnt about why some common coding practices came to be, such as writing "@Override" on code that overrides, naming conventions of classes and methods, as we were having difficulty trying to understand someone else's code at times.
Because of spending time trying to merge everyone's code together, we also learnt that that is why there is a head programmer or lead designer in a team, so that ideas flow in a logical direction and there won't be so many conflicts between components.
We learnt how to code with SOLID principles in mind, which can make code easier to edit. The principle of SRP, for example, is important and should be coded with the idea in mind so that when it comes to reading and understanding the code, it is much easier to identify and, as a result, becomes easier to debug. Another example is the DIP principle, which makes code easier to debug as well, since when we fix one portion of the code, we don't have to spend time looking elsewhere for the same issue and debug. We came to learn of this while trying to figure out each other's code and debugging.
We learnt the importance of camel casing and standardising them as it can make code easier to read and understand.

Future Improvements:

I think some improvements that could be made are implementing some trigger conditions that aren’t currently in the game into the code, such as when a combatant successfully hits another. It means that we can create more unique status effects that currently don't exist, leading to a more flexible and in-depth design with plenty of room to explore and create.

Another improvement could be in enemy behaviour. Currently, the only action that enemies are limited to is basic attacks. We could improve in such a way that instead, we call an action method, and the enemy implements the actions it wants to do. Thus, we can have enemies have multiple attacks or actions such as having basic attacks, defend and even a special attack as well, and the choices between them can be done using a random generator or a decision-making algorithm. This would make our code a bit more open and have more customisation of enemies for a more in-depth and flexible game.

Github Link

https://github.com/JabezThian/Java_OODP_SC2002.git

Game Test Cases

1. Damage Comparison: Normal vs. Defend

```
--- YOUR TURN: HeroA ---
  1. Basic Attack
  2. Special Skill: Shield Bash
  3. Defend (+10 DEF this round and next)
  4. Use Item
Choose an action: 1
Select target (1-2): 1
HeroA deals 25 damage to Goblin A
Goblin A now has 30 HP left.
[ENEMY TURN: Goblin A]
Goblin A deals 15 damage to HeroA
HeroA now has 245 HP left.
[ENEMY TURN: Goblin B]
Goblin B deals 15 damage to HeroA
HeroA now has 230 HP left.
```

```
--- YOUR TURN: HeroA ---
  1. Basic Attack
  2. Special Skill: Shield Bash
  3. Defend (+10 DEF this round and next)
  4. Use Item
Choose an action: 3
>> HeroA raises their shield! Defense increased by 10 for this round.
[ENEMY TURN: Goblin A]
Goblin A deals 5 damage to HeroA
HeroA now has 225 HP left.
[ENEMY TURN: Goblin B]
Goblin B deals 5 damage to HeroA
HeroA now has 220 HP left.
```

Normal attack: Warrior takes 15 damage from a Goblin.
After using Defend: Defense increases by 10, and the same Goblin deals only 5 damage.

2. Shield Bash Stun

```
--- YOUR TURN: HeroA ---
  1. Basic Attack
  2. Special Skill: Shield Bash
  3. Defend (+10 DEF this round and next)
  4. Use Item
Choose an action: 2
Select target for Shield Bash (1-2): 2
HeroA deals 25 damage to Goblin B
Goblin B now has 30 HP left.
[ENEMY TURN: Goblin A]
Goblin A deals 5 damage to HeroA
HeroA now has 215 HP left.
[ENEMY TURN: Goblin B]
>> Goblin B is stunned!


########################################
                ROUND 4
########################################

--- PARTY STATUS ---
[PLAYER] HeroA      | HP: 215/260 | ATK: 40  | DEF: 20 | SPD: 30
           Special [Shield Bash ]: Cooldown 2 turn(s)
           Items: Potion (x1), Power Stone (x1)

--- ENEMIES ---
1. Goblin A     | HP:  30/55  | ATK: 35  | DEF: 15 | SPD: 25
2. Goblin B     | HP:  30/55  | ATK: 35  | DEF: 15 | SPD: 25 [STUNNED]
TURN ORDER: HeroA (SPD 30) → Goblin A (SPD 25) → Goblin B (SPD 25)
```

Warrior uses Shield Bash on an enemy. The attack deals damage and applies the Stun status. The enemy is marked [STUNNED] and skips its next turn.

3. Potion Healing

```
--- YOUR TURN: HeroA ---
  1. Basic Attack
  2. [LOCKED] Special Skill (2 turn(s) left)
  3. Defend (+10 DEF this round and next)
  4. Use Item
Choose an action: 4

=== ITEMS ===
1. Potion (x1)
2. Power Stone (x1)
Select item number: 1
>> HeroA used Potion! Healed 45 HP.
>> HeroA now has 260/260 HP.
```

Warrior uses a Potion. HP is restored by 100, capped at the maximum health (260).

4. Smoke Bomb Damage Negation

```
--- YOUR TURN: HeroB ---
  1. Basic Attack
  2. Special Skill: Arcane Blast
  3. Defend (+10 DEF this round and next)
  4. Use Item
Choose an action: 4

=== ITEMS ===
1. Smoke Bomb (x1)
2. Potion (x1)
Select item number: 1
>> HeroB threw a Smoke Bomb!
>> Enemy attacks will deal 0 damage for 2 turns!
>> Smoke Bomb consumed. 0 remaining.

########################################
                ROUND 2
########################################

--- PARTY STATUS ---
[PLAYER] HeroB      | HP: 140/200 | ATK: 50  | DEF: 10 | SPD: 20
           Special [Arcane Blast]: READY
           Items: Potion (x1)
           Status: Smoke Bomb Invulnerability

--- ENEMIES ---
1. Goblin A     | HP:  55/55  | ATK: 35  | DEF: 15 | SPD: 25
2. Wolf A       | HP:  40/40  | ATK: 45  | DEF: 5  | SPD: 35
TURN ORDER: Wolf A (SPD 35) → Goblin A (SPD 25) → HeroB (SPD 20)

[ENEMY TURN: Wolf A]
Wolf A deals 0 damage to HeroB
HeroB now has 140 HP left.
[ENEMY TURN: Goblin A]
Goblin A deals 0 damage to HeroB
HeroB now has 140 HP left.
```

Warrior throws a Smoke Bomb. For the next two turns, all enemy attacks deal 0 damage. The status “Smoke Bomb Invulnerability” appears in the player’s status bar.

5. Wizard Arcane Blast – Attack Increase on Kill

```
#########################################
                ROUND 2
#########################################

--- PARTY STATUS ---
[PLAYER] HeroB      | HP: 140/200 | ATK: 50  | DEF: 10 | SPD: 20
           Special [Arcane Blast]: READY
           Items: Potion (x1)
           Status: Smoke Bomb Invulnerability
```

```
--- YOUR TURN: HeroB ---
  1. Basic Attack
  2. Special Skill: Arcane Blast
  3. Defend (+10 DEF this round and next)
  4. Use Item
Choose an action: 2
HeroB deals 35 damage to Goblin A
Goblin A now has 20 HP left.
HeroB deals 45 damage to Wolf A
Wolf A now has 0 HP left.
HeroB absorbed souls! Attack + 10

#########################################
                ROUND 3
#########################################

--- PARTY STATUS ---
[PLAYER] HeroB      | HP: 140/200 | ATK: 60  | DEF: 10 | SPD: 20
           Special [Arcane Blast]: Cooldown 2 turn(s)
           Items: Potion (x1)
           Status: Arcane Blast
```

Before: Wizard's ATK is 50.
After killing an enemy with Arcane Blast: ATK increases to 60. The bonus persists for the rest of the level.

6. Skill Cooldown

```
--- YOUR TURN: HeroB ---
  1. Basic Attack
  2. [LOCKED] Special Skill (2 turn(s) left)
  3. Defend (+10 DEF this round and next)
  4. Use Item
Choose an action: 2
>> ERROR: Arcane Blast is on cooldown! (2 turn(s) left)

--- YOUR TURN: HeroB ---
  1. Basic Attack
  2. [LOCKED] Special Skill (2 turn(s) left)
  3. Defend (+10 DEF this round and next)
  4. Use Item
Choose an action:
```

After using a special skill (Shield Bash or Arcane Blast), the skill enters a 3-turn cooldown. The menu shows [LOCKED] along with the remaining cooldown turns. Attempting to use it again displays an error message.

7. Backup Wave Spawn

```
--- YOUR TURN: HeroB ---
  1. Basic Attack
  2. [LOCKED] Special Skill (2 turn(s) left)
  3. Defend (+10 DEF this round and next)
  4. Use Item
Choose an action: 1
Select target (1-1): 1
HeroB deals 45 damage to Goblin A
Goblin A now has 0 HP left.

╔═══════ !! BACKUP WAVE INCOMING !! ═══════╗

  + Wolf B       | HP:  40 | ATK: 45  | DEF: 5  | SPD: 35
  + Wolf C       | HP:  40 | ATK: 45  | DEF: 5  | SPD: 35


########################################
                ROUND 4
########################################

--- PARTY STATUS ---
[PLAYER] HeroB      | HP: 115/200 | ATK: 60  | DEF: 10 | SPD: 20
           Special [Arcane Blast]: Cooldown 1 turn(s)
           Items: Potion (x1)
           Status: Arcane Blast

--- ENEMIES ---
1. Wolf B       | HP:  40/40  | ATK: 45  | DEF: 5  | SPD: 35
2. Wolf C       | HP:  40/40  | ATK: 45  | DEF: 5  | SPD: 35
TURN ORDER: Wolf B (SPD 35) → Wolf C (SPD 35) → HeroB (SPD 20)
```

When all initial enemies are defeated on Medium or Hard difficulty, a backup wave of enemies spawns immediately. The new enemies appear in the enemy list, and the battle continues.

Declaration Forms

Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment
We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course (SC2002/CE2002 CZ2002)	Lab Group	Signature /Date
THIAN ZHI XIN JABEZ (U2521525K)	SC2002	FCSB	Jabez 18/04/2026
TIMOTHY LOKE JUN KIT (U2522376L)	SC2002	FCSB	Timothy 18/04/2026
ZHANG HAOYUAN (U2521172F)	SC2002	FCSB	18/04/2026
ZHANG YUESHEN (U2520977L)	SC2002	FCSB	Zhang Yueshen 18/04/2026
TAN HONG WEI (U2522760L)	SC2002	FCSB	Hong Wei 18/04/2026

Important notes:

1. Name must **EXACTLY MATCH** the one printed on your Matriculation Card.
2. Student Code of Academic Conduct includes the latest guidelines on usage of Generative AI and any other guidelines as released by NTU.

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I **THIAN ZHI XIN JABEZ, jthian002@e.ntu.edu.sg** (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: **SC2002 OBJECT ORIENTED PROGRAMMING AND DESIGN**
2. Course Code: **SC2002**
3. Instructor: **ZHAO KESEN**
4. Title of Assignment/Project Submission: **SC2002 Group Assignment: Turn-Based Combat Arena**

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only. ☐
Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

✔

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
c. Apart from the foregoing notices, the submission is wholly my own work.

Thian Zhi Xin Jabez 18/04/2026
_____ Student Name & Signature Date

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I **Zhang Haoyuan**, **C250167@e.ntu.edu.sg** (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

5. Name of course: **SC2002 OBJECT ORIENTED PROGRAMMING AND DESIGN**
6. Course Code: **SC2002**
7. Instructor: **ZHAO KESEN**
8. Title of Assignment/Project Submission: **SC2002 Group Assignment: Turn-Based Combat Arena**

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only. ☐
Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

✔

I also declare that I have :

d. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
e. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
f. Apart from the foregoing notices, the submission is wholly my own work.

Zhang Haoyuan
ZH 18/04/2026 Student Name & Signature Date

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I **Zhang Yueshen**, **YZHANG350@e.ntu.edu.sg** honestly and sincerely make the following declaration in relation to the following course submission:

9. Name of course: **SC2002 OBJECT ORIENTED PROGRAMMING AND DESIGN**
10. Course Code: **SC2002**
11. Instructor: **ZHAO KESEN**
12. Title of Assignment/Project Submission: **SC2002 Group Assignment: Turn-Based Combat Arena**

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only. ☐
Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

✔

I also declare that I have :

g. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
h. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
i. Apart from the foregoing notices, the submission is wholly my own work.

Zhang Yueshen 18/04/2026 Student Name & Signature Date

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I **TIMOTHY LOKE JUN KIT, timo0044@e.ntu.edu.sg** (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

13. Name of course: **SC2002 OBJECT ORIENTED PROGRAMMING AND DESIGN**
14. Course Code: **SC2002**
15. Instructor: **ZHAO KESEN**
16. Title of Assignment/Project Submission: **SC2002 Group Assignment: Turn-Based Combat Arena**

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only. ☐
Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

✔

I also declare that I have :

j. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
k. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
l. Apart from the foregoing notices, the submission is wholly my own work.

Timothy Loke Jun Kit 18/04/2026
_____ Student Name & Signature Date

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I **TAN HONG WEI, htan171@e.ntu.edu.sg** honestly and sincerely make the following declaration in relation to the following course submission:

17. Name of course: **SC2002 OBJECT ORIENTED PROGRAMMING AND DESIGN**
18. Course Code: **SC2002**
19. Instructor: **ZHAO KESEN**
20. Title of Assignment/Project Submission: **SC2002 Group Assignment: Turn-Based Combat Arena**

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only. ☐
Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

✔

I also declare that I have :

m. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
n. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
o. Apart from the foregoing notices, the submission is wholly my own work.

Tan Hong Wei 18/04/2026
_____ Student Name & Signature Date